



Principios SOLID: el nombre de lo que ya se venía haciendo

El problema

La clase anterior terminó contando cuántos métodos de `Tienda` hubo que modificar para que `Indumentaria` entrara al catálogo. La respuesta fue cero. Ese cero no fue casualidad, fue el resultado de tres decisiones tomadas entre las clases 5 y 7: la jerarquía de `Producto`, el método abstracto `getPrecioFinal()` y un bucle que recorre `ArrayList<Producto>` sin preguntar de qué tipo es cada uno.

Del lado del local pasa lo mismo. El cierre de caja de la clase 7 recibe un `ArrayList<Pago>` y suma lo que se acredita. Ahora aparecen dos requerimientos nuevos. El contador pide que el cierre quede guardado en un archivo, y el dueño quiere recibir un resumen por correo al terminar el día. La primera versión que se escribe es la más directa, todo adentro de la clase que ya tenía el cierre:

```
public class Caja {  
  
    public double cerrar(ArrayList<Pago> pagos) {  
        double total = 0;  
        for (Pago pago : pagos) {  
            total += pago.montoAcreditado();  
        }  
        this.guardarEnArchivo(pagos, total);  
        this.enviarResumen(total);  
        return total;  
    }  
  
    private void guardarEnArchivo(ArrayList<Pago> pagos, double total) {  
        // abre cierre.txt, escribe una línea por pago y el total  
    }  
  
    private void enviarResumen(double total) {  
        ServicioCorreo correo = new ServicioCorreo("smtp.lapampa.com", 587);  
        correo.enviar("duenio@lapampa.com", "Cierre del día: $" + total);  
    }  
}
```

Funciona, y en una primera lectura no tiene nada raro. El costo se ve después. El contador cambia de sistema y ahora quiere el archivo separado por comas, y hay que abrir `Caja`. El dueño prefiere el resumen por mensaje en vez de por correo, y hay que abrir `Caja`. Alguien quiere probar que la suma está bien y descubre que cada prueba manda un correo real. Tres pedidos que no tienen nada que ver con sumar pagos, y los tres terminan en la misma clase.

No hay un bug. Hay una clase que va a tener que cambiar por motivos que no son suyos, y cada vez que se la toca se corre el riesgo de romper lo único que hacía bien.

Una forma de juzgar un diseño

Un diseño no se juzga por lo que hace hoy, se juzga por lo que va a costar el próximo cambio. Con ese criterio se escribieron, a lo largo de los años 90 y 2000, varios principios de diseño



orientado a objetos, y Robert C. Martin reunió cinco de ellos bajo la sigla SOLID. Cada letra es un principio, y cada principio describe una manera distinta en que un cambio puede volverse caro. La S es Single Responsibility, la O es Open/Closed, la L es Liskov Substitution, la I es Interface Segregation y la D es Dependency Inversion, y en ese orden se presentan a continuación.

No son reglas de sintaxis ni algo que el compilador verifique. Son criterios para decidir dónde va cada cosa. Y tres de los cinco, Open/Closed, Liskov Substitution e Interface Segregation, ya se aplicaron en este curso sin nombrarlos.

Single Responsibility

El enunciado dice que una clase debe tener una sola responsabilidad. Una responsabilidad es algo que la clase hace por cuenta de alguien, y la forma de contarlas es mirar quién las pide. Caja tiene tres: sumar el cierre, escribir el archivo y enviar el aviso. Y detrás de cada una hay una persona distinta: el local, el contador y el dueño.

La solución es repartir. Caja se queda con lo que sabe, sumar. El archivo pasa a una clase que solo sabe de archivos, y el envío a otra que solo sabe de envíos:

```
public class Caja {  
  
    public double cerrar(List<Pago> pagos) {  
        double total = 0;  
        for (Pago pago : pagos) {  
            total += pago.montoAcreditado();  
        }  
        return total;  
    }  
}  
  
public class ArchivoCierre {  
  
    public boolean guardar(List<Pago> pagos, double total) {  
        // abre cierre.txt, escribe una línea por pago y el total  
        return true;  
    }  
}  
  
public class ServicioCorreo {  
  
    public boolean enviar(String destinatario, String mensaje) {  
        // se conecta al servidor y manda el mensaje  
        return true;  
    }  
}
```

Ahora el pedido del contador toca ArchivoCierre y nada más. El del dueño toca ServicioCorreo. Y cerrar() se puede probar sin mandar un solo correo.

Este principio es el mismo criterio que apareció en la clase 3 con el encapsulamiento, cada clase es dueña de sus propias reglas. Lo que agrega es la forma de contar las responsabilidades: por quién las pide.



Una clase, una responsabilidad. Si dos partes de una clase responden a personas distintas, son dos.

Open/Closed

Este es el que ya se aplicó. El enunciado dice que una clase debe estar abierta a la extensión y cerrada a la modificación. Dicho así suena contradictorio, pero es exactamente lo que pasó con `Tienda` cuando entró `Indumentaria`, y lo que pasaría con `cerrarCaja` el día que aparezca `PagoTransferencia`. Se agrega un caso nuevo sin abrir el archivo donde el método está escrito.

Lo que lo hace posible es la combinación de abstracción y polimorfismo. El método depende de `Pago`, no de sus subclases, y el enlace dinámico se encarga de que cada objeto responda por sí mismo.

La señal de que el principio falta es conocida desde la clase 7, una cadena de `if` o de `instanceof` preguntando por el tipo. Cada vez que aparece un caso nuevo, esa cadena crece, y la clase que la contiene tiene que volver a abrirse.

Extender el sistema es agregar una clase, no editar las existentes.

Liskov Substitution

El polimorfismo de la clase 7 se apoyó en algo que no se dijo del todo. Cuando `cerrarCaja` llama a `pago.montoAcreditado()` sin saber qué tipo de pago tiene adelante, está confiando en que cualquier subclase va a comportarse como `Pago` promete. Si una sola no lo hace, el método deja de poder confiar en todas.

El principio lleva el apellido de Barbara Liskov, que lo formuló en 1987. Dice que un objeto de una subclase tiene que poder usarse en cualquier lugar donde se espera la superclase, sin que quien lo usa note la diferencia. Es más exigente que “compila”, porque el compilador solo verifica que los métodos existan, no lo que hacen.

La forma más común de romperlo es heredar una capacidad que la subclase no tiene. Supongamos que `anular()` se hubiera puesto en `Pago`. `PagoEfectivo` la hereda, y como en efectivo no hay nada que anular desde el sistema, alguien escribe esto:

```
@Override
public boolean anular() {
    throw new UnsupportedOperationException("El efectivo no se anula");
}
```

A partir de ahí, `anularTodo(Pago[] operaciones)` se cae con el primer pago en efectivo que encuentra, y quien lo llama empieza a escribir `if (pago instanceof PagoEfectivo)` antes de cada llamada. Se perdió Open/Closed por haber roto Liskov Substitution. En la clase 6 esa fue precisamente la razón para que `anular()` no fuera a `Pago` sino a la interfaz `Anulable`, que solo firman las clases que de verdad saben hacerlo.

La versión menos visible del mismo problema es una subclase que sobrescribe un método y le agrega condiciones que la superclase no tenía, o que devuelve valores que la superclase



nunca devolvería. Por ejemplo, un medio de pago cuyo `montoAcreditado()` devuelve un número negativo, algo que `Pago` nunca hizo y que `cerrarCaja` no está preparado para sumar. La pregunta para detectarlo es siempre la misma. Si en este lugar del programa se reemplaza el objeto por uno de la subclase, ¿algo deja de funcionar?

La subclase tiene que cumplir lo que la superclase promete, no solo compilar.

Interface Segregation

La clase 6 dejó dos interfaces separadas, `Anulable` y `Exportable`. La alternativa hubiera sido una sola, `Operacion`, con `anular()`, `estaAnulado()` y `exportar()` juntas. El problema aparece con `PagoEfectivo`, que se exporta al archivo del cierre pero no se anula. Para cumplir `Operacion` tendría que escribir un `anular()` que no hace nada o que lanza una excepción, y eso es otra vez la subclase que miente.

El enunciado dice que ninguna clase debería verse obligada a depender de métodos que no usa. Es mejor tener varias interfaces chicas, una por capacidad, que una grande que junte todo. Cada clase implementa solo las interfaces que puede cumplir, y el código que usa `Anulable` no se entera de que además existe `Exportable`.

La biblioteca de Java está armada así. `Comparable` tiene un método, `Runnable` tiene uno, `Iterable` tiene uno. Y la señal de que este principio falta es fácil de ver, un método implementado vacío o con un `throw` solo para que la clase compile.

Ninguna clase debería depender de métodos que no usa.

Dependency Inversion

Después de repartir responsabilidades queda un cabo suelto. El cierre de caja sigue teniendo que avisar al dueño, eso no cambió. Lo que salió de `caja` fue el conocimiento de *cómo* se manda un correo, que ahora vive en `ServicioCorreo`. Pero alguien tiene que llamar a `ServicioCorreo` cuando termina el cierre, y lo más directo es que lo haga `Caja`:

```
public class Caja {  
  
    private final ServicioCorreo correo = new ServicioCorreo("smtp.lapampa.com", 587);  
  
    public double cerrar(List<Pago> pagos) {  
        double total = 0;  
        for (Pago pago : pagos) {  
            total += pago.montoAcreditado();  
        }  
        this.correo.enviar("duenio@lapampa.com", "Cierre del dia: $" + total);  
        return total;  
    }  
}
```

`caja` es una clase de alto nivel, expresa una regla del negocio. `ServicioCorreo` es un detalle, la forma en que hoy se manda un aviso. Y la clase de alto nivel depende del detalle, con nombre, servidor y puerto incluidos. El día que el aviso vaya por mensaje, `caja` cambia. Para probar `cerrar()`, hace falta un servidor de correo.



El enunciado dice que los módulos de alto nivel no deben depender de los de bajo nivel, y que ambos deben depender de abstracciones. En código, es una interfaz definida desde el punto de vista de quien la necesita, y un constructor que la recibe:

```
public interface Notificador {
    boolean enviar(String mensaje);
}

public class Caja {

    private final Notificador notificador;

    public Caja(Notificador notificador) {
        if (notificador == null) {
            throw new IllegalArgumentException("El notificador es obligatorio");
        }
        this.notificador = notificador;
    }

    public double cerrar(List<Pago> pagos) {
        double total = 0;
        for (Pago pago : pagos) {
            total += pago.montoAcreditado();
        }
        this.notificador.enviar("Cierre del día: $" + total);
        return total;
    }
}
```

ServicioCorreo pasa a escribir implements Notificador, y el día que haga falta aparece ServicioMensajes implements Notificador, sin que Caja se entere. Para las pruebas se escribe un NotificadorDePrueba que solo guarda el último mensaje recibido.

La palabra “inversión” viene de mirar quién depende de quién. Antes, caja dependía de ServicioCorreo. Ahora las dos dependen de Notificador, y esa interfaz la define la necesidad de caja, no la existencia del servicio de correo.

Queda una pregunta. ¿Dónde se escribe el new ServicioCorreo(...)? En algún lado tiene que estar. La respuesta es afuera de Caja, en main o en la parte del programa que arma los objetos al arrancar. A esa forma de entregar las dependencias por constructor se la llama inyección de dependencias, y es lo que List<String> nombres = new ArrayList<>() de la clase 6 dejaba entrever, programar contra la interfaz, con el detalle concreto en una sola línea. Por eso las firmas de esta clase reciben List<Pago> y no ArrayList<Pago>: a cerrar() no le importa cómo está guardada la lista, y quien la llama puede cambiarla sin abrir Caja.

El detalle se adapta a la regla, y no al revés.

Los cinco, juntos

Principio	La pregunta que hace	Señal de que falta
Single Responsibility	¿Cuántas responsabilidades	Una clase que suma, escribe



Principio	La pregunta que hace	Señal de que falta
	tiene esta clase?	archivos y manda correos
Open/Closed	¿Qué hay que abrir para agregar un caso?	Un <code>if</code> o <code>instanceof</code> encadenado por tipo
Liskov Substitution	¿Se puede poner la subclase donde se espera la superclase?	Un método heredado que lanza “no soportado”
Interface Segregation	¿Obliga la interfaz a implementar lo que no se usa?	Un método vacío para que la clase compile
Dependency Inversion	¿Depende de una interfaz o de un detalle?	Un <code>new</code> de una clase concreta adentro de otra

Los cinco se apoyan en los mismos dos mecanismos, la abstracción y el polimorfismo. No son cinco técnicas distintas, son cinco maneras de mirar el mismo diseño y preguntarle cuánto va a costar cambiarlo.

También conviene decir lo que no son. No son una lista para aplicar por adelantado en cada clase. Una clase con tres atributos y dos métodos no necesita una interfaz propia ni un constructor que reciba tres dependencias. El principio se aplica cuando el cambio aparece o cuando es muy probable que aparezca, y el criterio es siempre el costo de ese cambio, no la cantidad de clases que tiene el proyecto.

La clase que viene, la interfaz gráfica, va a ser el primer lugar donde estos criterios se aplican de entrada, porque la lógica del programa no tiene que saber que hay una pantalla.

Bibliografía sugerida para profundizar

Martin, R. C. (2002). *Agile Software Development, Principles, Patterns, and Practices*. Prentice Hall.

- Capítulos 8 a 12, uno por principio y en este orden: Single Responsibility, Open/Closed, Liskov Substitution, Dependency Inversion e Interface Segregation. Es el texto original, con los ejemplos de los que salen casi todos los que circulan.

Martin, R. C. (2017). *Clean Architecture*. Prentice Hall.

- Parte III, capítulos 7 a 11. La misma presentación, más corta y con veinte años de perspectiva. El capítulo 7 reformula Single Responsibility en términos de actores, cada responsabilidad responde a una persona distinta, que es el criterio que se usa en esta clase para contarlas.

Deitel, P., & Deitel, H. (2017). *Java How to Program, Early Objects*.



- Capítulo 10, para el mecanismo que sostiene a los cinco, el polimorfismo con clases abstractas e interfaces.